COMSATS University Islamabad
Abbottabad Campus

Department of Computer Science

SUBJECT:

SOFTWARE QUALITY ENGINEERING

LAB TASK.

SUBMITTED TO:

SIR MUKHTIAR ZAMIN

SUBMITTED BY:

AIMAN SHABBIR **FA22-BSE-104**

HAMZA ALI **FA23-BSE-142**

SECTION:

BSE – 6B

DATE:

17-6-2026.

1. Introduction to the Tool

TestNG with Java is an automation testing framework used to execute and manage test cases automatically. It provides features such as annotations, assertions, test grouping, parallel execution, and reporting. TestNG is widely used with Selenium for automating web applications and can also be used for API testing and performance testing. IntelliJ IDEA serves as the development environment for writing and executing TestNG scripts.

2. Related Concepts and Definitions

TestNG

TestNG (Test Next Generation) is a Java-based testing framework used for creating and executing automated tests efficiently.

Automation Testing

Automation testing is the process of using software tools to execute test cases automatically and compare actual results with expected results.

Assertions

Assertions are statements used to validate whether the actual output matches the expected output.

Annotations

Annotations are predefined keywords provided by TestNG to control the execution flow of test methods.

Common annotations include:

- @Test
- @BeforeMethod
- @AfterMethod
- @BeforeClass
- @AfterClass

Web Testing

Web testing verifies the functionality, usability, and behavior of web applications.

Common HTTP Request Methods:

- GET
- POST

- PUT
- DELETE

API Testing

API testing validates communication between different components of an application by checking requests, responses, status codes, and returned data.

Performance Testing

Performance testing evaluates the speed, responsiveness, and stability of an application under various workloads.

Common performance testing tools:

- Apache JMeter
- LoadRunner

3. Setup Procedure

Tools Required

- Java JDK
- IntelliJ IDEA
- Maven
- TestNG Framework

Setup Steps

1. Install Java JDK.
2. Install IntelliJ IDEA.
3. Create a Maven project.
4. Add the TestNG dependency in the pom.xml file.
5. Install the TestNG plugin in IntelliJ IDEA.
6. Build the project and configure TestNG.
7. Create a test class and add test methods.
8. Execute the tests using TestNG.

4. Sample Running Tests

Login Feature Test Cases

Feature

User logs in using:

- Username (UserName field)
- Password
- Login button

Positive Test Case

Test Case 1: Valid Login

Steps

1. Open login page.
2. Enter valid username.
3. Enter valid password.
4. Click Login button.

Test Data

- UserName: valid.user@example.com
- Password: Test@12345

Expected Result

- User should successfully login.
- Dashboard should open.
- No error message should appear.

Negative Test Cases

Test Case 2: Invalid Username

Steps

1. Enter wrong username.
2. Enter correct password.
3. Click Login.

Test Data

- UserName: wrong.user@example.com
- Password: Test@12345

Expected Result

- Error message should appear: "Invalid credentials".

Test Case 3: Invalid Password

Steps

1. Enter correct username.
2. Enter wrong password.
3. Click Login.

Test Data

- UserName: valid.user@example.com
- Password: wrongpass

Expected Result

- Error message: "Invalid credentials".

Test Case 4: Empty Username

Steps

1. Leave username empty.
2. Enter password.
3. Click Login.

Expected Result

- Validation message: "Username is required".

Test Case 5: Empty Password

Steps

1. Enter username.
2. Leave password empty.

3. Click Login.

Expected Result

- Validation message:
  "Password is required".

Test Case 6: Both Fields Empty

Steps

1. Leave username empty.
2. Leave password empty.
3. Click Login.

Expected Result

- Validation message:
  "Fields cannot be empty".

Test Case 7: Special Characters in Username

Steps

1. Enter @@@### in username.
2. Enter password.
3. Click Login.

Expected Result

- Error message or validation rejection.

Test Case 8: SQL Injection Attempt (Security Test)

Steps

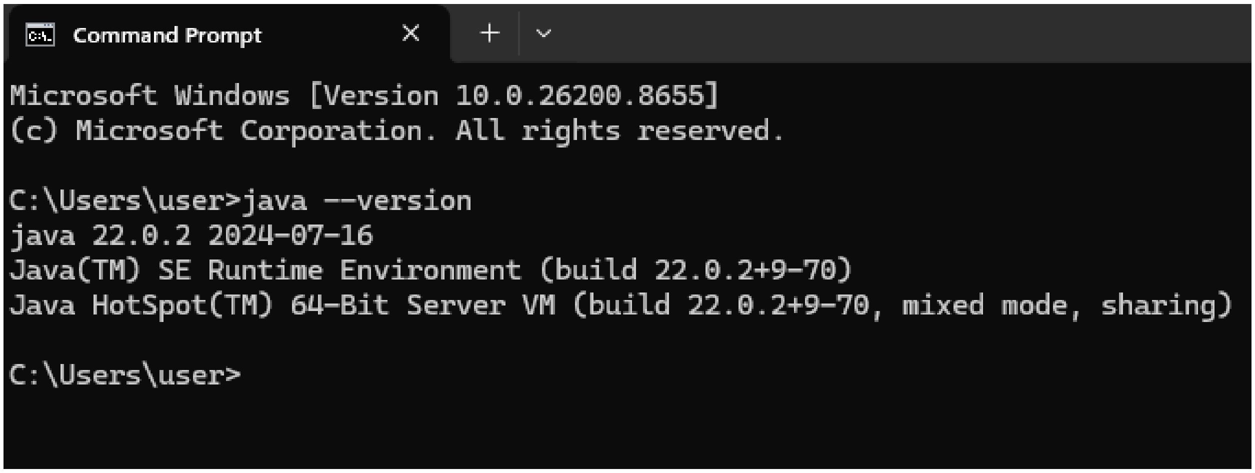
1. Enter username: ' OR '1'='1
2. Enter any password.
3. Click Login.

Expected Result

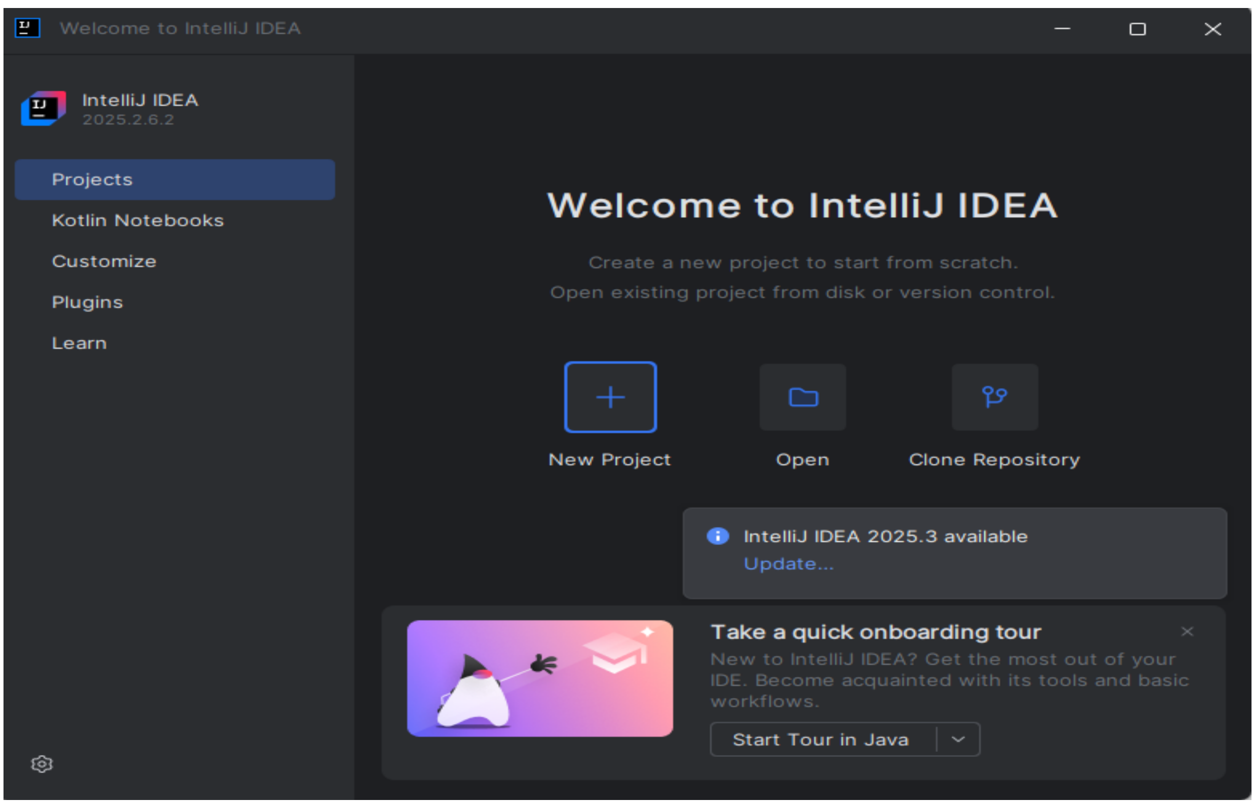
- Login should fail.
- System should not accept SQL injection.

Steps

Step 1: Install Java JDK

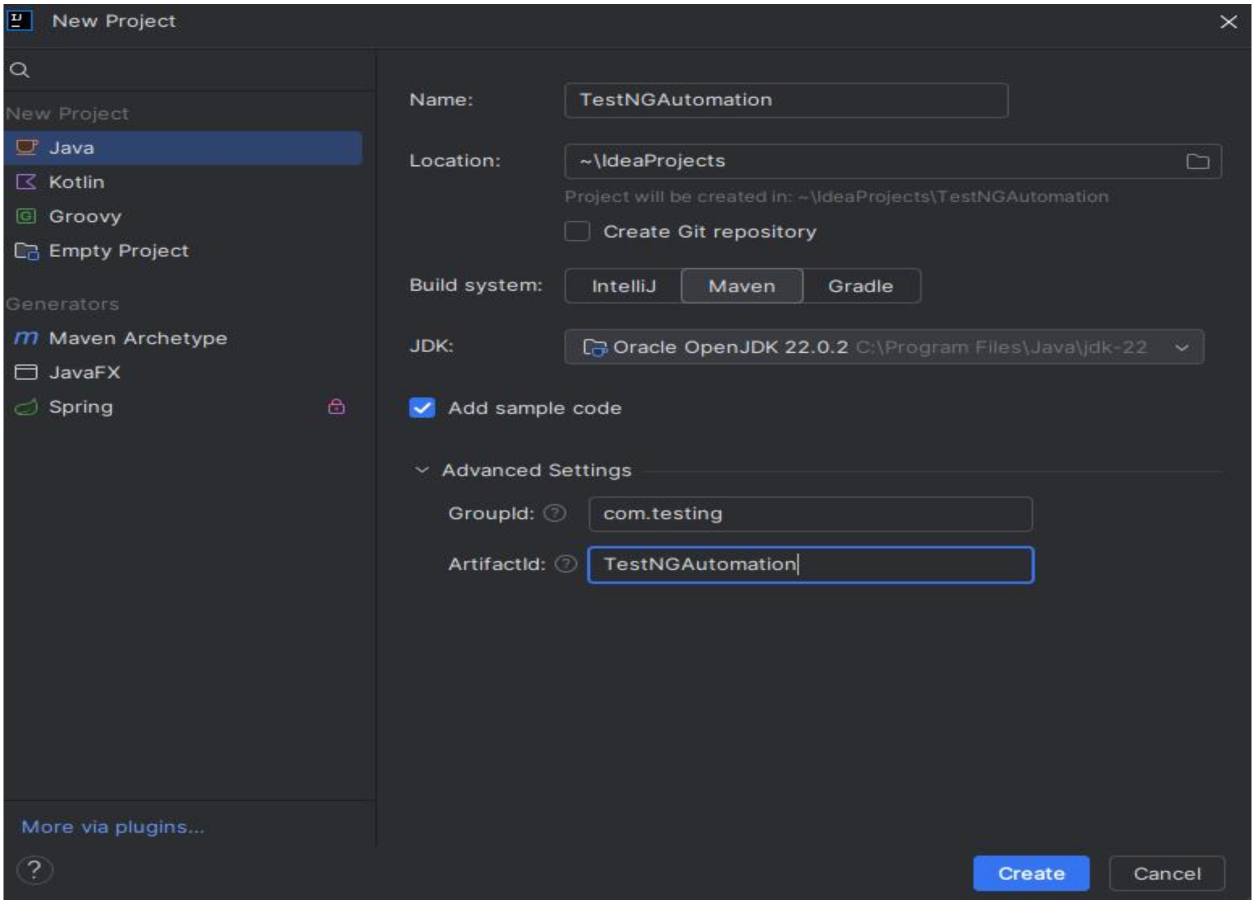


Step 2: Install IntelliJ IDEA



Step 3: Create a Maven Project in IntelliJ IDEA

Creating a project



Pom . xml file

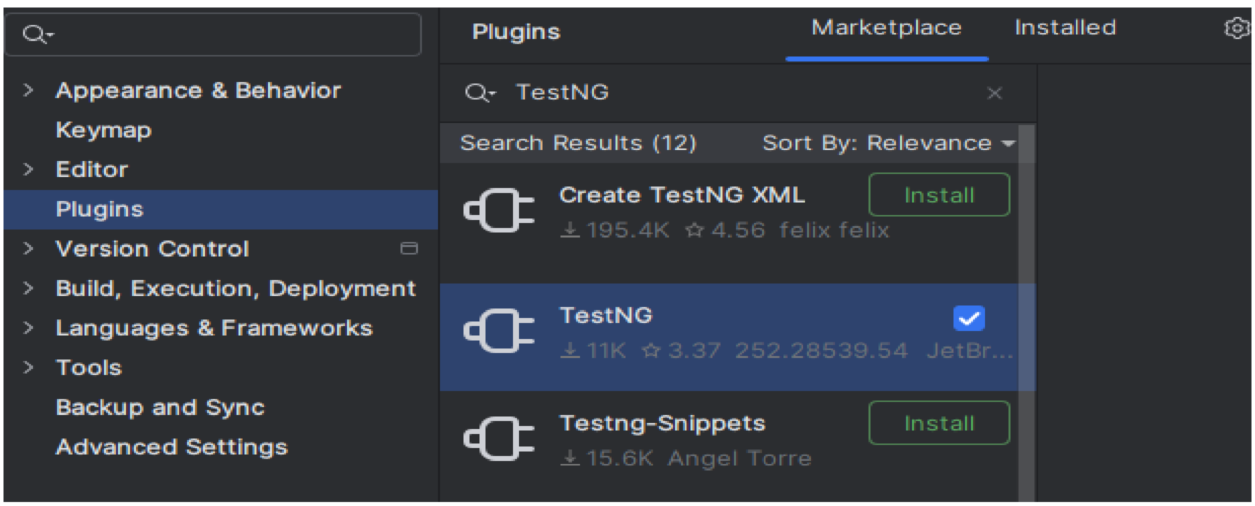
```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.testing</groupId>
    <artifactId>TestNGAutomation</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>22</maven.compiler.source>
        <maven.compiler.target>22</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

</project>
```

Step 4: Add TestNG Dependency

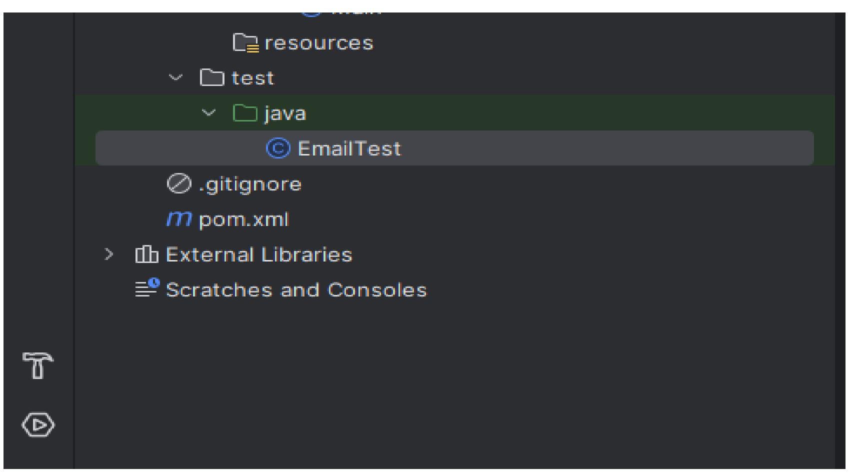
```
    </properties>
    <dependencies>

        <dependency>
            <groupId>org.testng</groupId>
            <artifactId>testng</artifactId>
            <version>7.10.2</version>
            <scope>test</scope>
        </dependency>

    </dependencies>

</project>
```

Step 5: Install TestNG Plugin in IntelliJ IDEA

TestNG is installed



Step 6: Create a Test Class

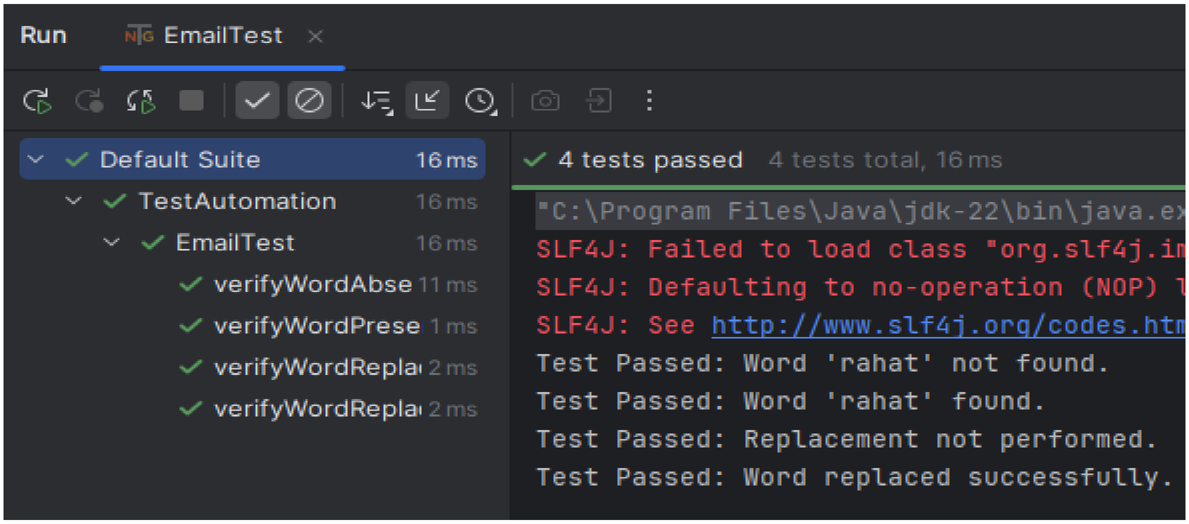


Step 7: Write Your Test

```java
import org.testng.annotations.Test;
Rename usages
public class EmailTest {
    // Positive Scenario
    @Test
    public void verifyWordPresent() {
        String emailContent =
                "Hello, welcome to the system. Thank you, rahat.";
        Assert.assertTrue(emailContent.contains("rahat"));
        System.out.println("Test Passed: Word 'rahat' found.");
    }
    // Negative Scenario
    @Test
    public void verifyWordAbsent() {
        String emailContent =
                "Hello, welcome to the system. Thank you, user.";
        Assert.assertFalse(emailContent.contains("rahat"));
        System.out.println("Test Passed: Word 'rahat' not found.");
    }
    // Positive Replacement Test
    @Test
    public void verifyWordReplacementSuccess() {
        String emailContent = "Hello user";
        String modifiedContent =
                emailContent.replace( target: "user",  replacement: "rahat");
        Assert.assertTrue(modifiedContent.contains("rahat"));
```

```java
        Assert.assertFalse(modifiedContent.contains("user"));
        System.out.println("Test Passed: Word replaced successfully.");
    }
    // Negative Replacement Test
    @Test
    public void verifyWordReplacementFailure() {
        String emailContent = "Hello user";
        // No replacement performed
        String modifiedContent = emailContent;
        Assert.assertFalse(modifiedContent.contains("rahat"));
        Assert.assertTrue(modifiedContent.contains("user"));
        System.out.println("Test Passed: Replacement not performed.");
    }
}
```

Step 8: Run the Test

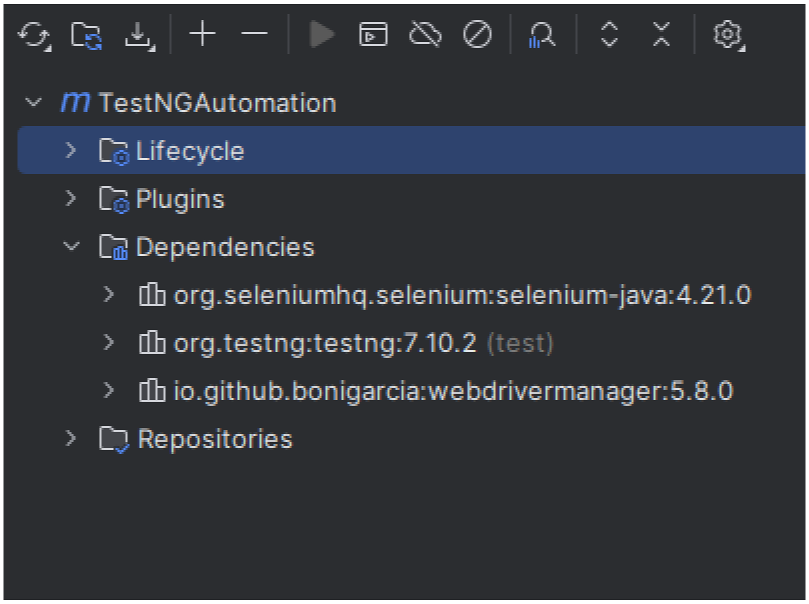


Scenario Opening gmail

Pom.xml

```xml
    </properties>
    <dependencies>
        <!-- Selenium WebDriver -->
        <dependency>
            <groupId>org.seleniumhq.selenium</groupId>
            <artifactId>selenium-java</artifactId>
            <version>4.21.0</version>
        </dependency>
        <!-- TestNG -->
        <dependency>
            <groupId>org.testng</groupId>
            <artifactId>testng</artifactId>
            <version>7.10.2</version>
            <scope>test</scope>
        </dependency>
        <!-- WebDriverManager -->
        <dependency>
            <groupId>io.github.bonigarcia</groupId>
            <artifactId>webdrivermanager</artifactId>
            <version>5.8.0</version>
        </dependency>
    </dependencies>
</project>
```

Dependencies

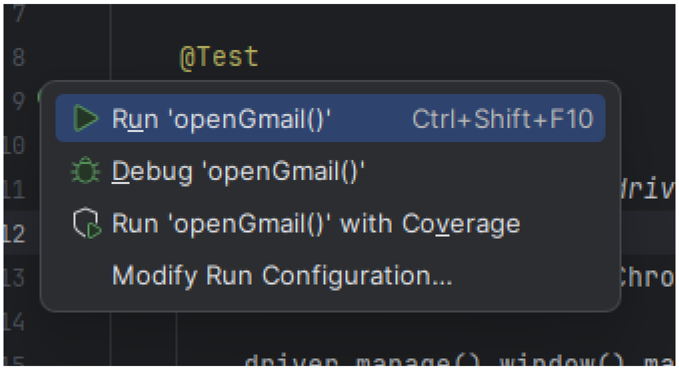


Gmail test

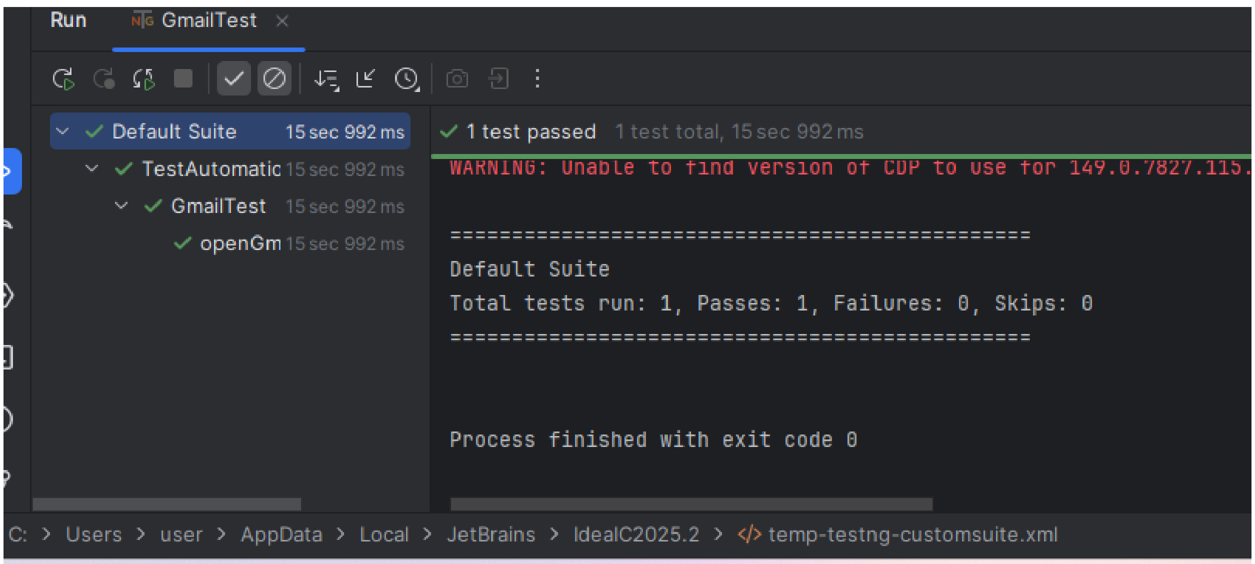
Code

```java
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.annotations.Test;
public class GmailTest {
    @Test
    public void openGmailAndEnterEmail() throws InterruptedException {
        WebDriver driver = new ChromeDriver();
        driver.manage().window().maximize();
        driver.get("https://mail.google.com/");
        Thread.sleep(3000);
        driver.findElement(By.id("identifierId"))
                .sendKeys("test.user@example.com");
        Thread.sleep(5000);
        driver.quit();
    }
}
```

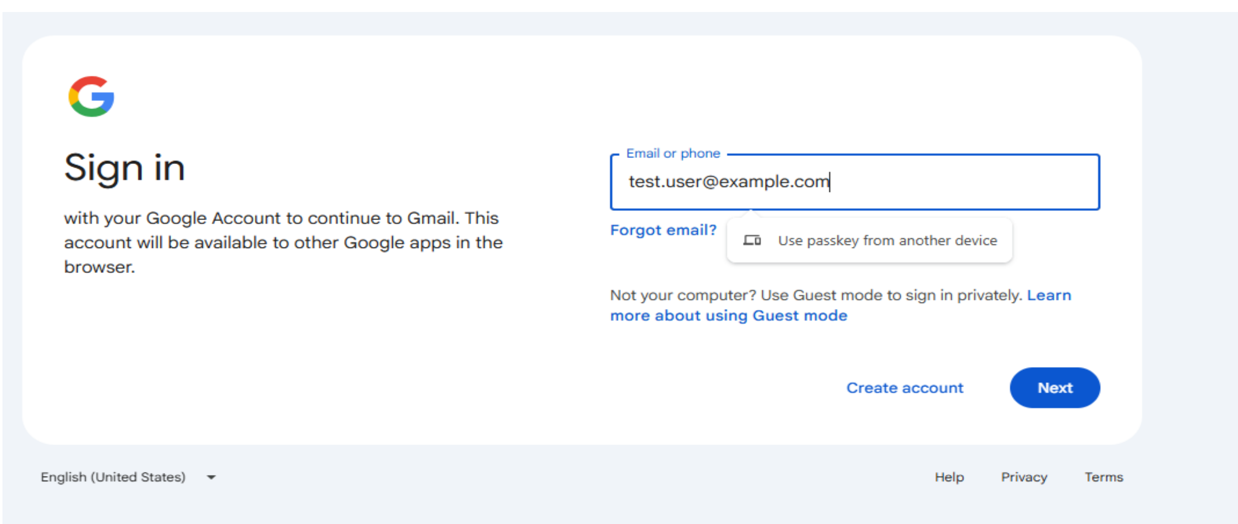
Right click on open Gmail and then run the open Gmail



Output



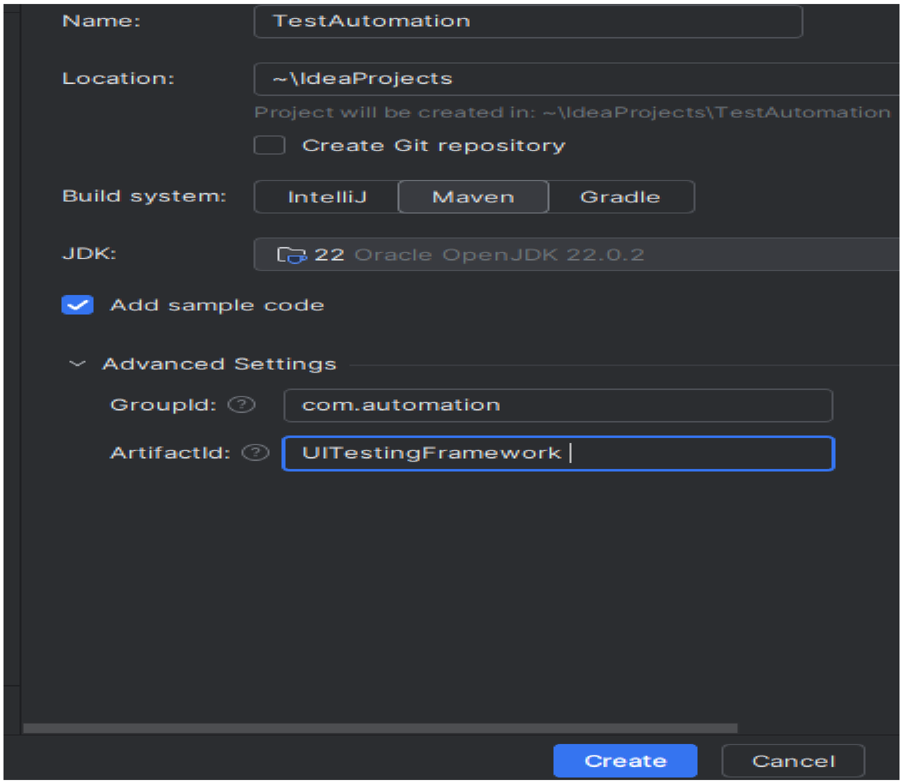
Gmail Open successfully



Login Subject Expert Scenario

Step 1: Environment Setup (IntelliJ IDEA + TestNG + Java + Selenium)

Creating new project



Add dependencies

```xml
    </properties>
    <dependencies>

        <!-- Selenium -->
        <dependency>
            <groupId>org.seleniumhq.selenium</groupId>
            <artifactId>selenium-java</artifactId>
            <version>4.20.0</version>
        </dependency>

        <!-- TestNG -->
        <dependency>
            <groupId>org.testng</groupId>
            <artifactId>testng</artifactId>
            <version>7.10.2</version>
            <scope>test</scope>
        </dependency>

    </dependencies>

</project>
```

Maven reload

Right side **Maven tab**

Click **Reload All Maven Projects**

<u>Step 2: Scenario Design (UI Automation Testing)</u>

Scenario: "Subject Expert Website – Login Functionality"

Application Type:

Educational Web Application (Subject Expert Portal)

Main Features:

1. User Login
2. Dashboard Access

Test Scenario Flow

Scenario Name:

Verify user login functionality

Preconditions:

- Website should be accessible
- Browser should be opened

<u>Step 3: Selenium + TestNG Automation (Login Test)</u>

Login Feature – Full Test Cases

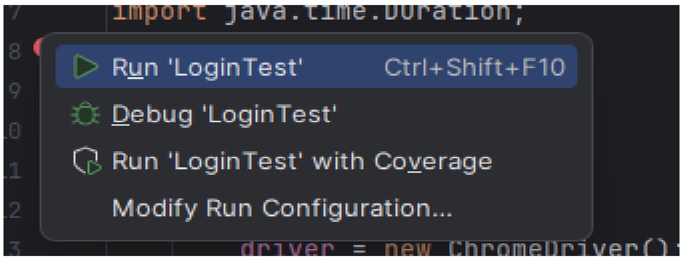
Code:

```java
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.WebDriverWait;
import org.testng.Assert;
import org.testng.annotations.*;
import java.time.Duration;
public class LoginTest {
    WebDriver driver;  33 usages
    WebDriverWait wait;  1 usage
    @BeforeMethod
    public void setup() {
        driver = new ChromeDriver();
        driver.manage().window().maximize();
        driver.get("https://yourwebsite.com/login"); // replace with real URL
        wait = new WebDriverWait(driver, Duration.ofSeconds(10));
    }
    // ========================
    // POSITIVE TEST CASE
    // ========================
    @Test
    public void validLoginTest() {
        driver.findElement(By.id("UserName")).sendKeys( ...keysToSend: "valid.user@example.com");
        driver.findElement(By.id("Password")).sendKeys( ...keysToSend: "Test@12345");
        driver.findElement(By.id("LoginButton")).click();
        // validation after login
        String actualTitle = driver.getTitle();
```

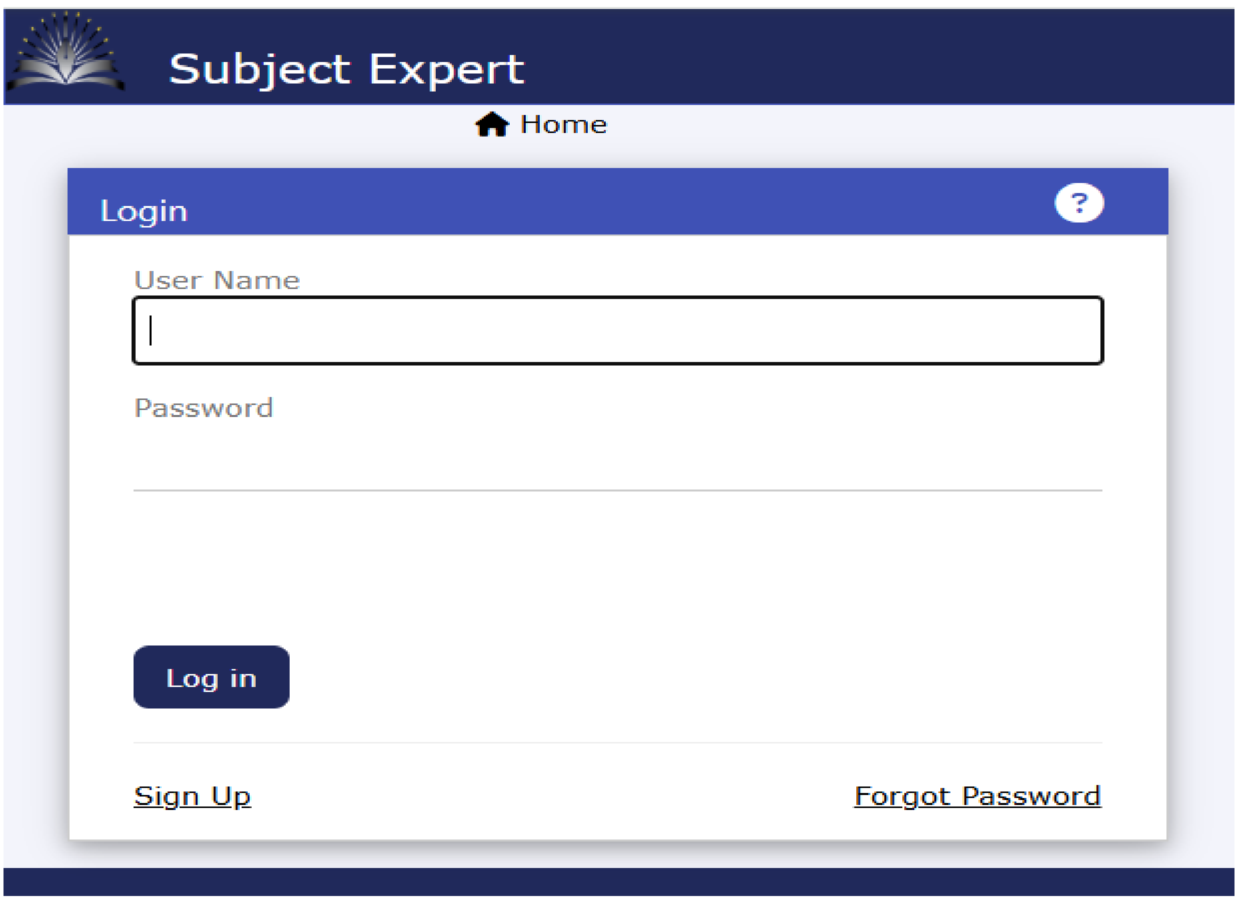
```java
        Assert.assertTrue(actualTitle.contains("Dashboard"),  message: "Login failed!");
    }
    // ========================
    // INVALID USERNAME
    // ========================
    @Test
    public void invalidUsernameTest() {
        driver.findElement(By.id("UserName")).sendKeys( ...keysToSend: "wrong.user@example.com");
        driver.findElement(By.id("Password")).sendKeys( ...keysToSend: "Test@12345");
        driver.findElement(By.id("LoginButton")).click();
        String error = driver.findElement(By.id("errorMessage")).getText();
        Assert.assertEquals(error,  expected: "Invalid credentials");
    }
    // ========================
    // INVALID PASSWORD
    // ========================
    @Test
    public void invalidPasswordTest() {
        driver.findElement(By.id("UserName")).sendKeys( ...keysToSend: "valid.user@example.com");
        driver.findElement(By.id("Password")).sendKeys( ...keysToSend: "wrongpass");
        driver.findElement(By.id("LoginButton")).click();
        String error = driver.findElement(By.id("errorMessage")).getText();
        Assert.assertEquals(error,  expected: "Invalid credentials");
    }
    // ========================
    // EMPTY USERNAME
```

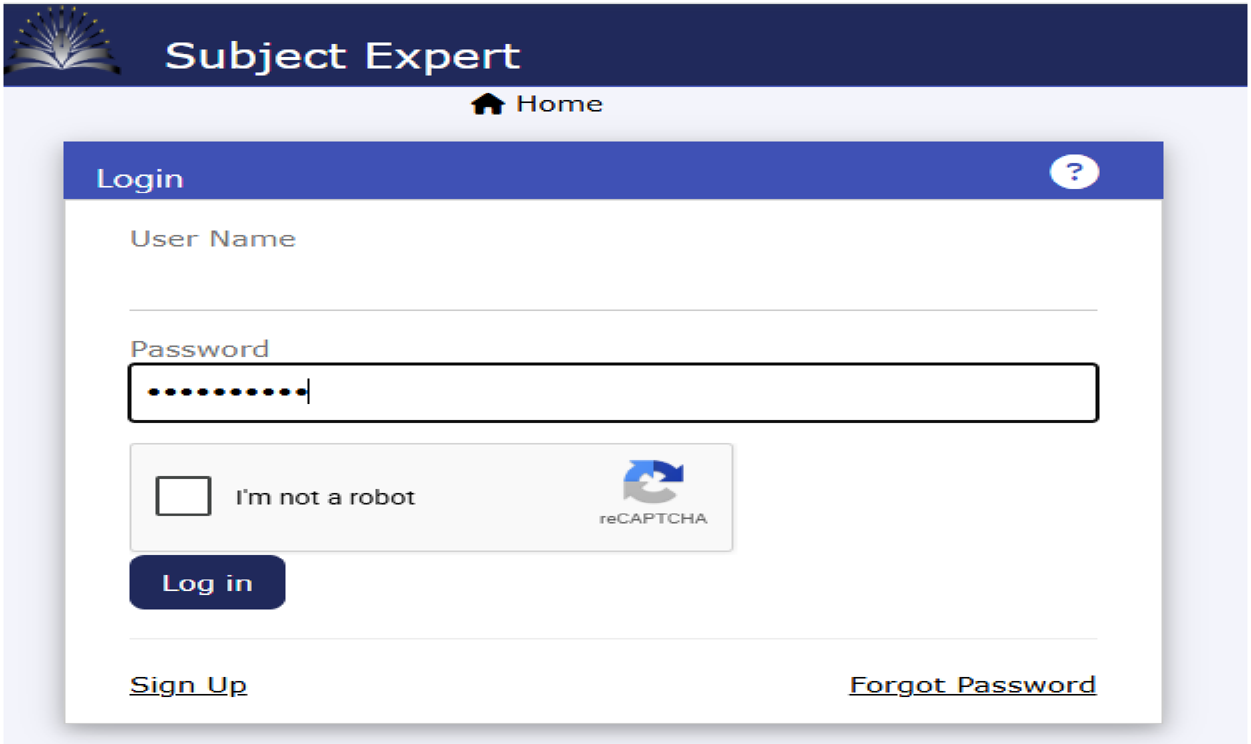
```java
    // EMPTY USERNAME
    // ========================
    @Test
    public void emptyUsernameTest() {
        driver.findElement(By.id("Password")).sendKeys( ...keysToSend: "Test@12345");
        driver.findElement(By.id("LoginButton")).click();
        String msg = driver.findElement(By.id("validationMessage")).getText();
        Assert.assertEquals(msg,  expected: "Username is required");
    }
    // ========================
    // EMPTY PASSWORD
    // ========================
    @Test
    public void emptyPasswordTest() {
        driver.findElement(By.id("UserName")).sendKeys( ...keysToSend: "valid.user@example.com");
        driver.findElement(By.id("LoginButton")).click();
        String msg = driver.findElement(By.id("validationMessage")).getText();
        Assert.assertEquals(msg,  expected: "Password is required");
    }
    // ========================
    // BOTH EMPTY
    // ========================
    @Test
    public void emptyFieldsTest() {
        driver.findElement(By.id("LoginButton")).click();
        String msg = driver.findElement(By.id("validationMessage")).getText();
        Assert.assertEquals(msg,  expected: "Fields cannot be empty");
    }
    // ========================
    // SPECIAL CHARACTERS
    // ========================
    @Test
    public void specialCharUsernameTest() {
        driver.findElement(By.id("UserName")).sendKeys( ...keysToSend: "@@@###");
        driver.findElement(By.id("Password")).sendKeys( ...keysToSend: "Test@12345");
        driver.findElement(By.id("LoginButton")).click();
        String error = driver.findElement(By.id("errorMessage")).getText();
        Assert.assertTrue( condition: error.length() > 0);
    }
    // ========================
    // SQL INJECTION TEST
    // ========================
    @Test
    public void sqlInjectionTest() {
        driver.findElement(By.id("UserName")).sendKeys( ...keysToSend: "' OR '1'='1");
        driver.findElement(By.id("Password")).sendKeys( ...keysToSend: "any");
        driver.findElement(By.id("LoginButton")).click();
        String error = driver.findElement(By.id("errorMessage")).getText();
        Assert.assertEquals(error,  expected: "Invalid credentials");
    }
    @AfterMethod
    public void tearDown() throws InterruptedException {
```

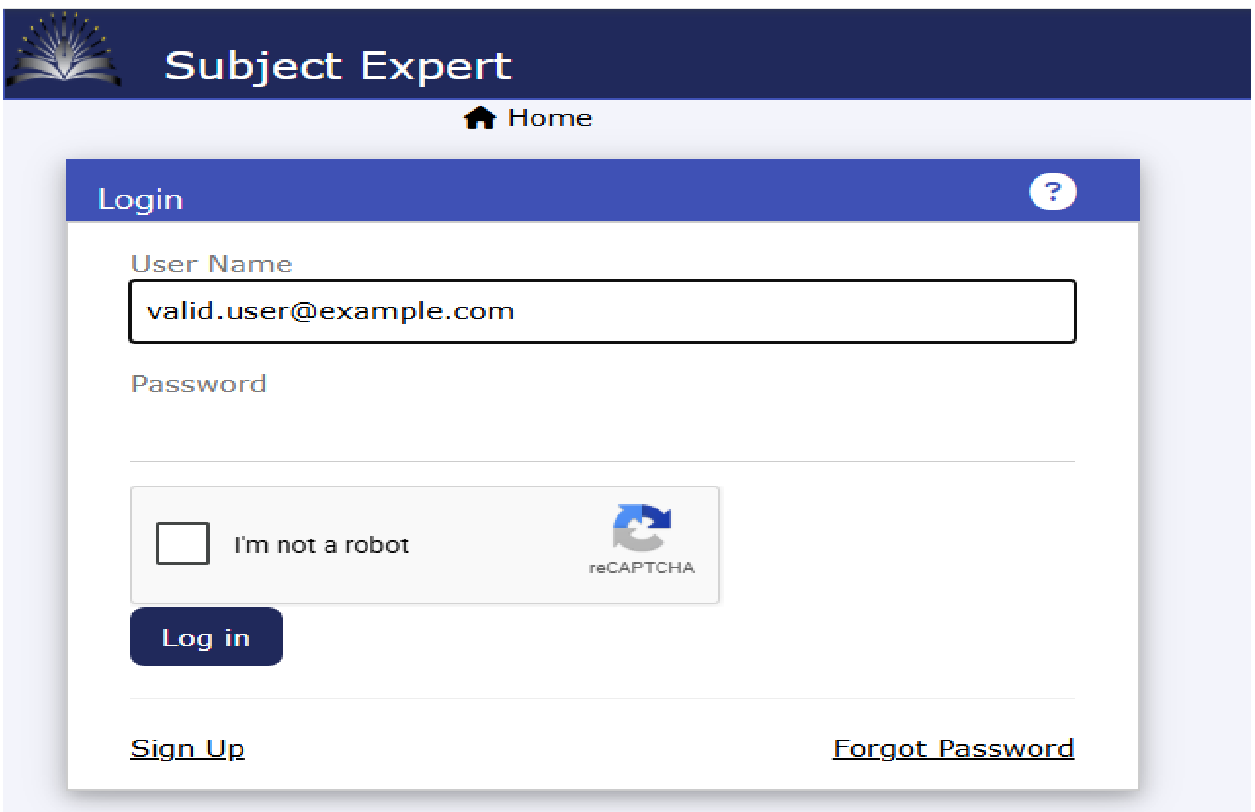
```
    @AfterMethod
    public void tearDown() throws InterruptedException {
        Thread.sleep( millis: 2000);
        driver.quit();
    }
}
```

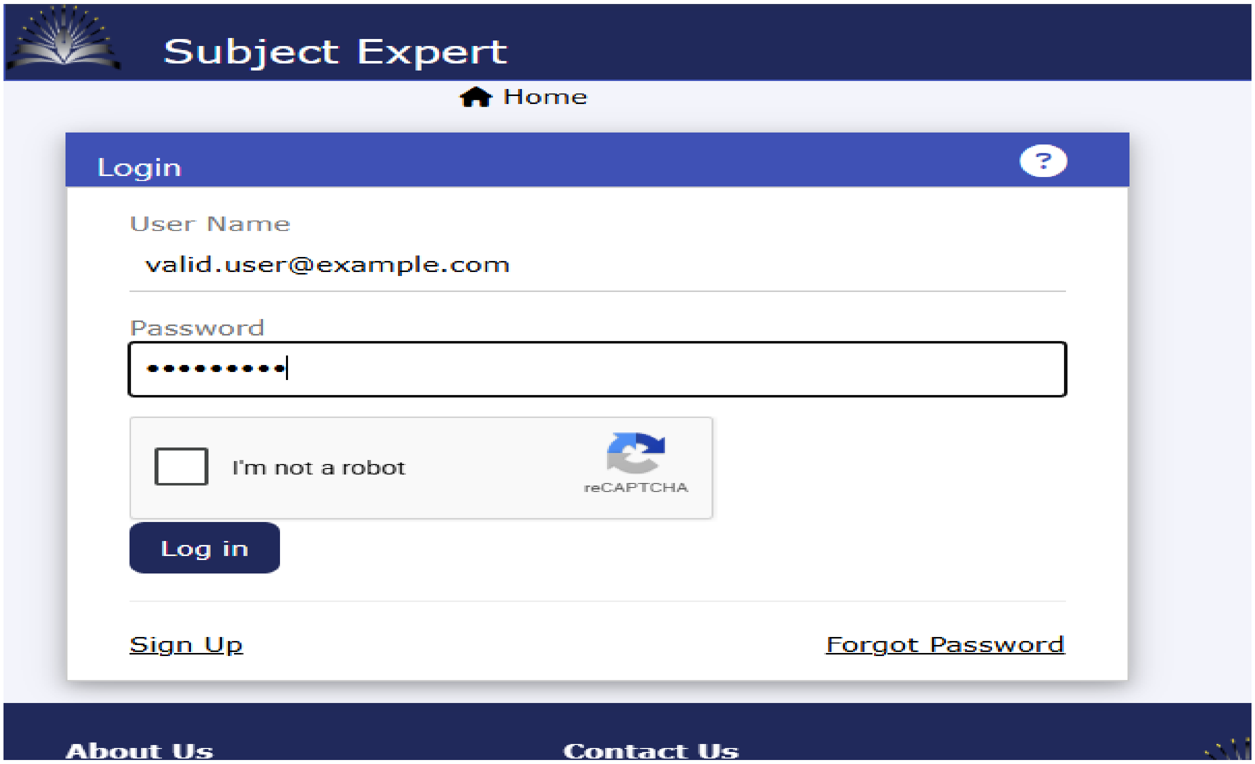


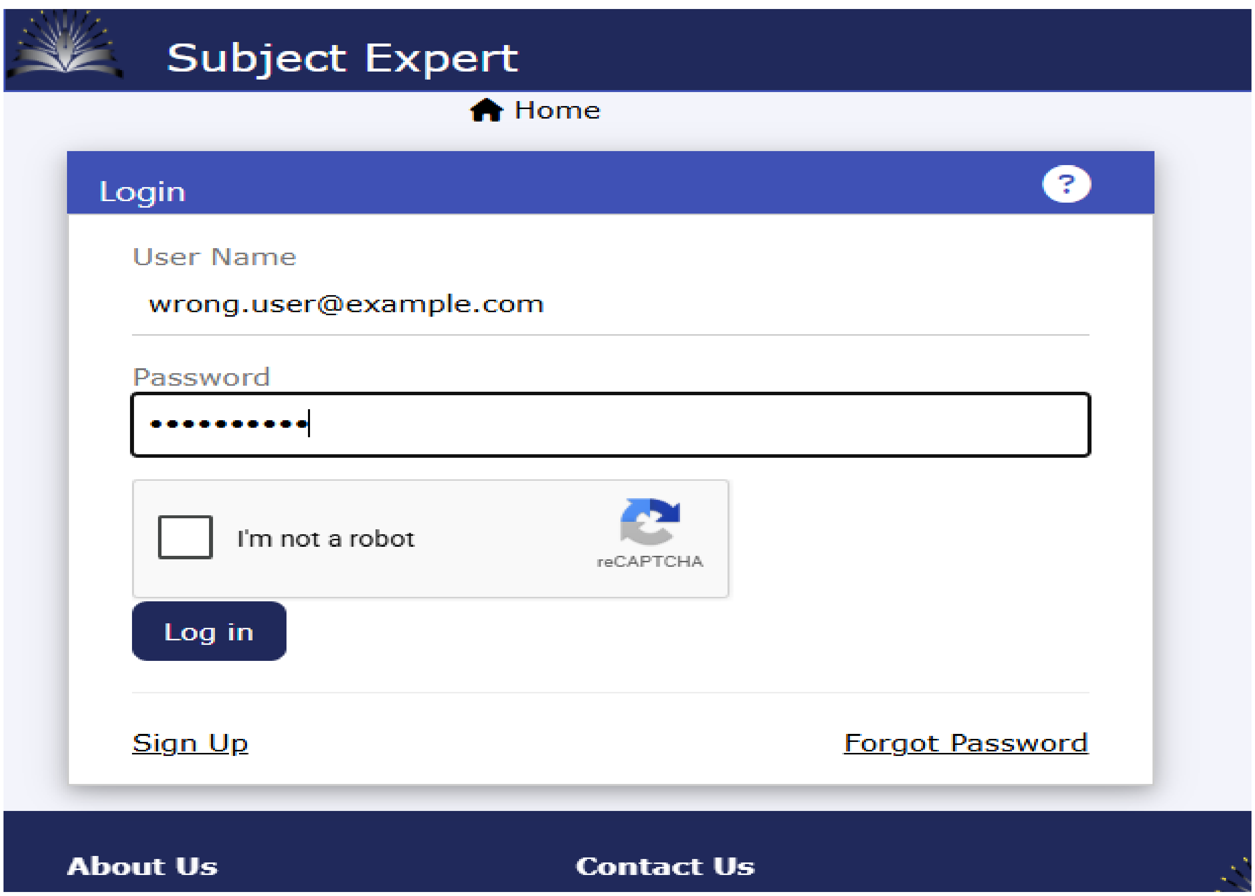
Output:

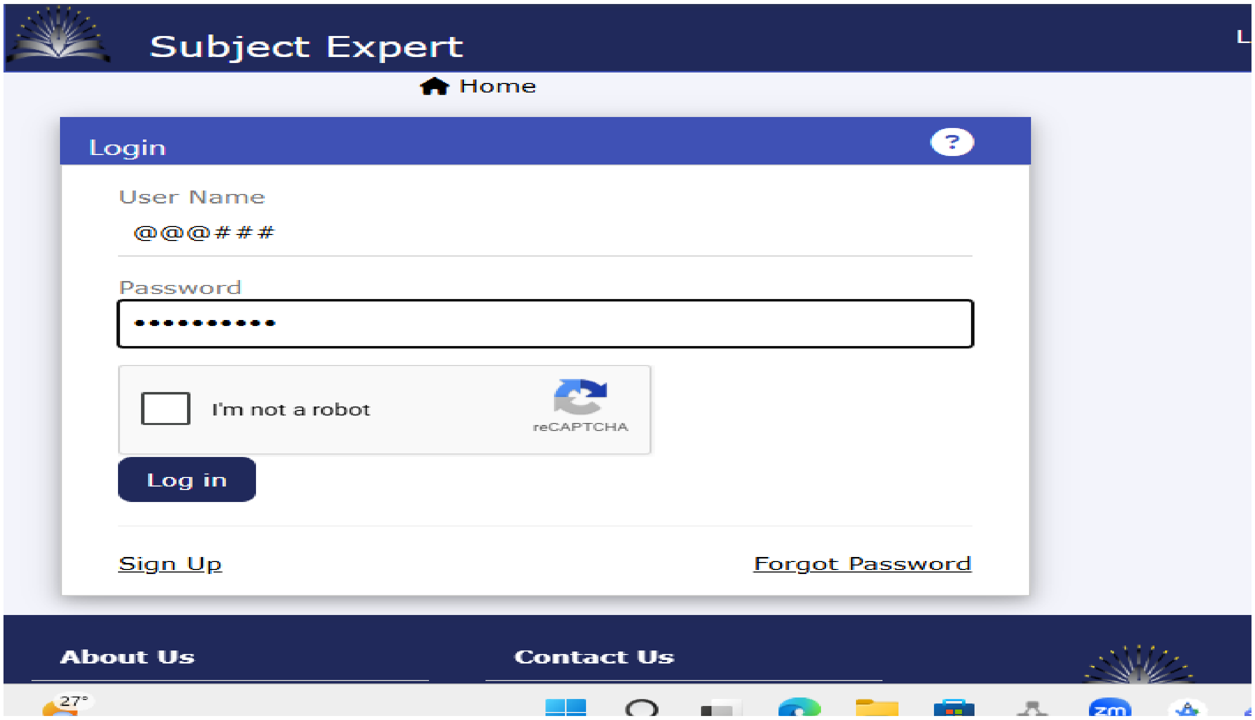


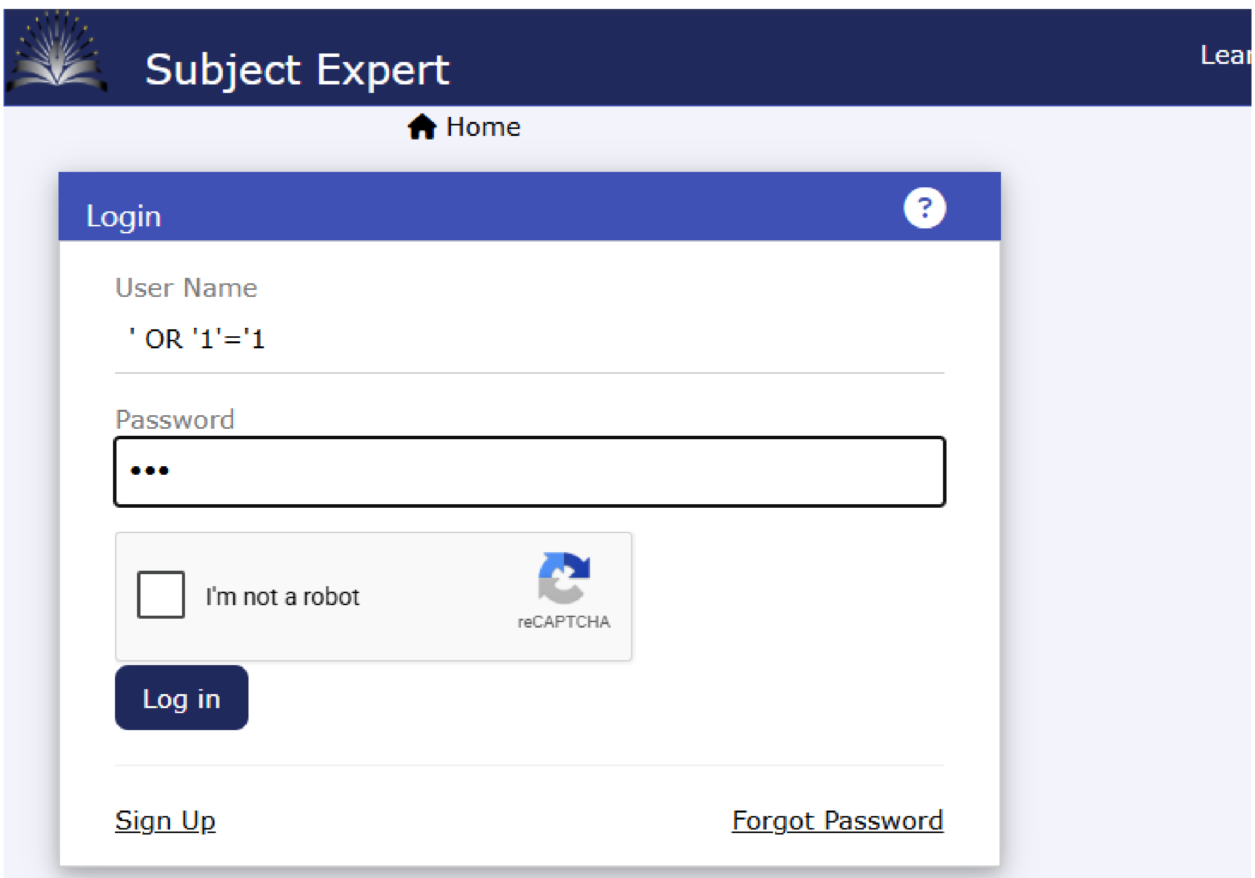

Subject Expert
Home
Login
?
User Name
Password
I'm not a robot
reCAPTCHA
Log in
Sign Up
Forgot Password

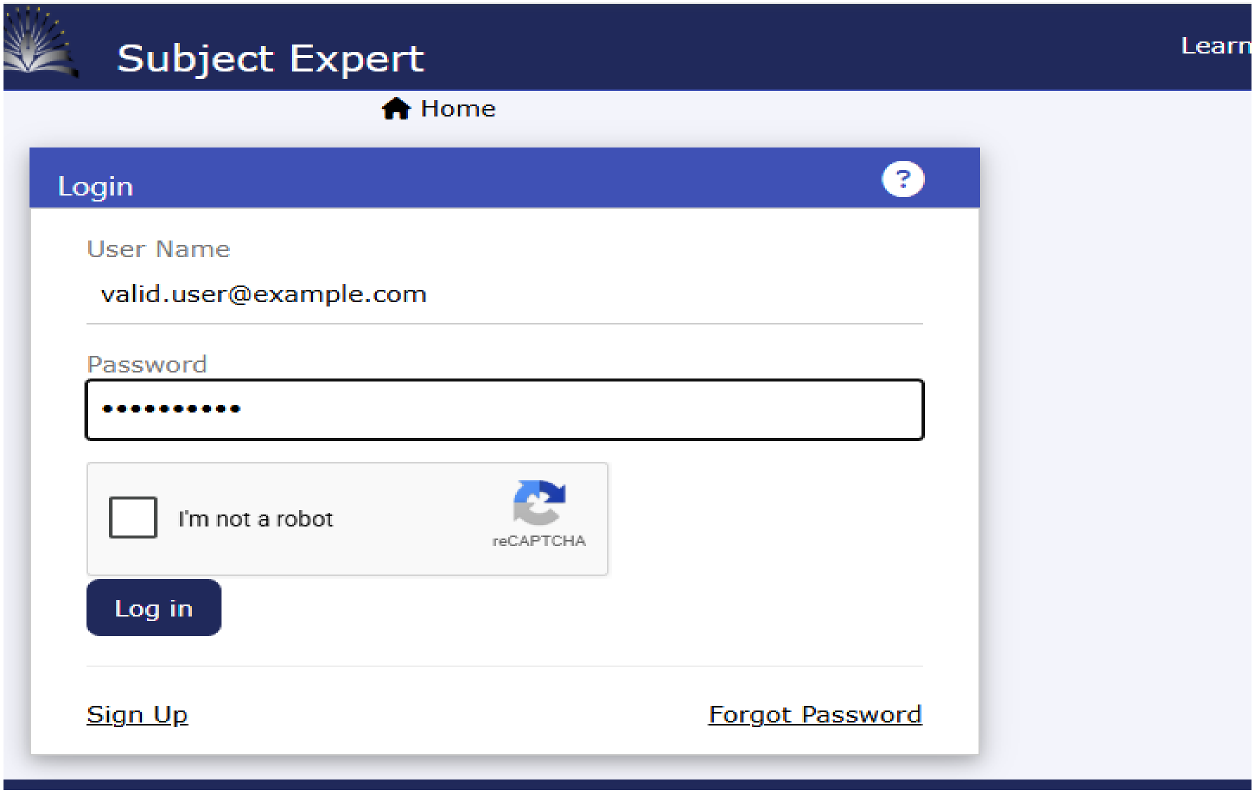

Subject Expert
Home
Login
?
User Name
valid.user@example.com
Password
I'm not a robot
reCAPTCHA
Log in
Sign Up
Forgot Password


Subject Expert
Home
Login
User Name
valid.user@example.com
Password
I'm not a robot
reCAPTCHA
Log in
Sign Up
Forgot Password
About Us
Contact Us


Subject Expert
Home
Login
User Name
wrong.user@example.com
Password
I'm not a robot
reCAPTCHA
Log in
Sign Up
Forgot Password
About Us
Contact Us


Subject Expert
Home
Login
User Name
@@@###
Password
I'm not a robot
reCAPTCHA
Log in
Sign Up
Forgot Password
About Us
Contact Us
27°


Subject Expert
Lear
Home
Login
User Name
' OR '1'='1
Password
I'm not a robot
reCAPTCHA
Log in
Sign Up
Forgot Password




```
===============================================
Default Suite
Total tests run: 8, Passes: 0, Failures: 8, Skips: 0
===============================================


Process finished with exit code 0
```